ByteBoost Workshop: Project Description

Shape-Constrained Boosted Symbolic Regression for Interpretable Neural Scaling Laws: A Cross-Testbed Investigation

---

1 Field of science

Computer Science: Machine Learning and Artificial Intelligence, specifically the foundations of large language models (neural scaling laws) and interpretable machine learning via symbolic regression. Secondary field: High-Performance Computing and computer architecture (cross-platform performance characterization).

2 Abstract / summary

Training a modern AI language model can cost millions of dollars, so researchers use scaling laws to predict performance from model size and training data before committing resources. These equations guide major hardware and budget decisions, but their mathematical form is usually chosen manually and then fitted to measured results.

Our project uses symbolic regression to discover the scaling-law equation directly from data. Symbolic regression searches many candidate formulas and returns an interpretable equation rather than a black-box model. Unconstrained searches, however, often produce formulas that fit existing measurements but extrapolate unrealistically, predicting negative error or worse performance with more data.

We therefore require every candidate formula to satisfy admissibility axioms: larger models and additional data cannot increase error; improvements must diminish with scale; error must approach a positive irreducible floor; excess loss must decay as a power law; and the formula must remain smoothly well-defined down to the smallest scales of interest. The discovery method is soft shape-constrained search: a genetic-programming fitness *penalizes* continuum axiom violations via interval-arithmetic scores, drives them toward zero, and bounds any remaining shortfall (Sections 4.2 and 4.4 to 4.6 and Proposition 3).

The formula is built incrementally from a standard scaling law, adding one correction at a time while preserving all guarantees (Propositions 1 and 2 and Theorem 1). We also prove that the corrections improve fit without changing the irreducible error floor or the asymptotic rate of improvement whenever the soft scores vanish (Theorem 1 and Proposition 3).

The project has two computational bottlenecks. Model training will use Cerebras CS-3 systems on PSC's Neocortex platform; new validation losses will be compared to already-published loss curves from an existing scaling grid on Hugging Face (Sections 3.1 and 3.2). Formula discovery is a large CPU-bound search dominated by interval-arithmetic certification. Because it is highly parallel and relies mainly on scalar or imperfectly vectorized Python (with optional JAX acceleration), Stony Brook's high-core-count AMA27 AmpereOne cluster is well suited to the workload (Sections 3.1 and 4.4).

Deliverables will include a certified scaling-law formula fitted to our training data, an open-source discovery library, and a concise methods report (Section 5.2). An initial set of trained models is already public (Section 3.5).

3 Key requirements

3.1 Testbed hardware (requested)

Code skeleton: `src/modeling/training/`, `src/systems/hpc/`, `src/search/`

- **Neocortex (PSC):** Cerebras CS-3 wafer-scale systems for the model-pretraining grid in Sections 3.4 and 3.5 (`https://www.psc.edu/resources/neocortex/`). Each CS-3 is built around a Wafer Scale Engine 3 (WSE-3) with on-chip SRAM and extreme memory bandwidth, so a model in our N range can stay on a single wafer without multi-GPU model parallelism. We use Neocortex only for pretraining; the resulting validation losses are compared to the already-published loss curves of Section 3.2 (not to re-running those models on student-accessible GPUs).
- **AMA27 (Stony Brook Supercomputing Center / ACCESS):** Arm-based AmpereOne A192-32M cluster for the CPU-bound symbolic-regression search of Section 4.3 and the certification inner loop of Section 4.4. Per the ACCESS resource description, phase 1 deploys 312 single-socket compute nodes, each with a 192-core AmpereOne A192-32M CPU and 384 GB DDR5, linked by 200 Gbps NDR InfiniBand, with a 5 PB GPFS filesystem (`https://support.access-ci.org/affinity-groups/ama27`). AMA27 is the Stony Brook Arm testbed featured in ByteBoost 2.0 (in place of the retired Ookami A64FX system). We do *not* rely on A64FX-style 512-bit SVE or HBM: the GP/IA workload is population-parallel and largely scalar Python DualInterval evaluation, so the useful AMA27 properties are Arm ISA diversity, high core counts per node, and dense node count for concurrent candidate evaluation.

3.2 Existing scaling-law baselines (public artifacts)

Code skeleton: `src/scaling/data/scaling_dataset.py`, `src/modeling/training/trainer.py`

An already-public `colinear_scaling_models` collection on Hugging Face [1] provides trained LLM checkpoints and the associated validation-loss curves from a prior scaling grid. Those artifacts are the workshop baseline: students compare *losses* $L(\mathbf{h})$ (and the scaling laws fit to them), not the original training hardware, which is neither part of this project nor available to participants. New Neocortex pretraining runs should emit losses on a comparable protocol so they can be plotted and analyzed against that public loss inventory (Sections 3.4 and 3.5).

3.3 Software and tools

Code skeleton: `src/modeling/training/`, `src/search/`, `src/constraints/certificates/`

PyTorch 2.x for pretraining (distributed data-parallel training; optional mixed precision), with `uv` for environment management and Weights & Biases for experiment logging [2]. Symbolic search uses `gplearn` [3] with a custom fitness that adds interval-arithmetic axiom penalties, the sole discovery method of Section 4.5; an optional PySR backend provides an unconstrained alternate engine for ablation. Certificates are evaluated with a pure-Python forward-mode *DualInterval* interval-arithmetic layer (Section 4.4); JAX with custom JVPs is an optional acceleration path, not required.

3.4 Models

Code skeleton: `src/modeling/models/config.py`, `src/modeling/models/transformer.py`

Decoder-only transformer language models (a compact post-LN causal stack suitable for scaling grids), spanning roughly 10M–1.4B parameters, across several tokens-per-parameter ratios ($M = D/N$).

Workshop ports may adopt LLaMA-style blocks [4], FlashAttention, and μP / μTransfer [5]; the core method only requires consistent (N, D, L) measurements in the Step Law [6] / overtraining [7] regimes.

3.5 Datasets

Code skeleton: `src/scaling/data/scaling_dataset.py`, `src/scaling/data/corpora.py`

Pretraining corpora used to generate new grid points include Wikipedia [8], RedPajama [9], and related open text mixtures (optional FineWeb-Edu [10] for workshop extensions). An already-public `colinear_scaling_models` collection on Hugging Face [1] supplies trained checkpoints and loss curves for the symbolic-regression track and for loss-level comparison with new Neocortex runs (Section 3.2); the original training hardware is out of scope.

4 Method summary (technical)

Code skeleton: `src/scaling/`, `src/constraints/`, `src/search/`; map: Appendix B

Symbols used below are collected in Appendix A; the student skeleton layout is summarized in Appendix B.

4.1 Setup

Code skeleton: `src/scaling/setup/configuration.py`, `src/scaling/setup/constants.py`

Let N be the parameter count, D the training-token count, and $\mathcal{H} = \{h_1, \dots, h_m\}$ a finite set of remaining hyperparameters (in the concrete 4D grid used here, weight decay and learning rate); see also Appendix A. For each coordinate $h \in \{N, D\} \cup \mathcal{H}$, let H_h be its finite set of admissible values and $\phi_h \colon H_h \to \mathbb{R}_{>0}$ a strictly monotone preprocessing map; write $\phi_i \coloneqq \phi_{h_i}$ for brevity. In the search implementation, genetic-programming features are the logarithms $\log \phi_h(h)$ (so stage corrections are fit in log-coordinates and mapped back to raw-scale derivatives by the chain rule; Section 4.4). The configuration space is the product $\mathbb{H} \coloneqq \prod_{h \in \{N,D\} \cup \mathcal{H}} H_h$, with typical element $\mathbf{h} \in \mathbb{H}$ (so N, D, h_i denote the corresponding coordinates of $\mathbf{h}$). Let $L \colon \mathbb{H} \to \mathbb{R}$ be validation loss; we seek $\hat{L} \approx L$. We are given a labeled dataset $\mathcal{D} = \{(\mathbf{h}_\ell, L(\mathbf{h}_\ell))\}_{\ell=1}^{n} \subset \mathbb{H} \times \mathbb{R}$. For each scale variable $x \in \{N, D\}$ set $x_{\min} \coloneqq \min H_x > 0$ and $x_{\max} \coloneqq \max H_x$, treat x as continuous on $[x_{\min}, \infty)$, and take all derivatives with the other coordinates held fixed. Write $\widetilde{\mathbb{H}}$ for the corresponding continuum domain: each scale ranges over $[x_{\min}, \infty)$ while non-scale coordinates still range over their finite H_h, so $\mathbb{H} \subset \widetilde{\mathbb{H}}$. The labeled loss L is observed only on $\mathbb{H}$; the approximant $\hat{L}$ is evaluated on $\widetilde{\mathbb{H}}$ when stating shape constraints (Section 4.2).

4.2 Admissibility axioms

Code skeleton: `src/constraints/axioms/admissibility.py`

$\hat{L}$ is *admissible* ($\hat{L} \in \mathcal{S}$) if it satisfies (A1)–(A5) below. Where an axiom mentions a scale variable x, it is imposed for each $x \in \{N, D\}$ with the other coordinates held fixed. Although the labeled grid $\mathbb{H}$ is finite, $\hat{L}$ is constrained as a real function on the continuum domain $\widetilde{\mathbb{H}}$ of Section 4.1, so that the derivatives in (A1)–(A2) and the limits in (A3)–(A4) are well-defined.

(A1) **Monotone decrease** [11, 12]. $\dfrac{\partial \hat{L}}{\partial x} \le 0$ for all $x \ge x_{\min}$.

(A2) **Diminishing returns** [11, 12]. $\dfrac{\partial^2 \hat{L}}{\partial x^2} \geq 0$ for all $x \geq x_{\min}$. (Together with (A1), this is the usual diminishing-returns shape: further increases in x yield weakly smaller loss reductions. The direction is worth stating, because diminishing returns is classically drawn as a *concave* curve: that picture is of a gain, whereas $\hat{L}$ is a loss, and a gain such as $\hat{L}(x_{\min}, \cdot) - \hat{L}$ is concave exactly when $\hat{L}$ is convex. Everything below certifies the convex form, which is why the score of Section 4.5 is called v_{conv}.)

(A3) **Irreducible loss** [11, 13]. There is a positive *joint floor* L_∞ and, along each axis, a *marginal floor* L_x^∞:

(i) *Joint floor.* There exists $L_\infty > 0$ such that $\hat{L}(\mathbf{h}) > L_\infty$ for every $\mathbf{h} \in \widetilde{\mathbb{H}}$.

(ii) *Marginal floor.* For each $x \in \{N, D\}$ and every fixed choice of the other coordinates, the single-axis limit

$$L_x^\infty \;\coloneqq\; \lim_{t \to \infty} \hat{L}(\mathbf{h})\big|_{x=t} \tag{1}$$

exists and is finite, and satisfies $L_x^\infty \geq L_\infty$. (The value L_x^∞ depends on the frozen coordinates; the notation suppresses that dependence.)

In the stage-0 Chinchilla law, L_∞ equals the joint limit as all scales $\to \infty$ (namely E), while a single-axis marginal is in general strictly larger. Joint-floor positivity $L_\infty > 0$ already forces $\hat{L} > 0$ on $\widetilde{\mathbb{H}}$, so a separate positivity axiom is unnecessary.

(A4) **Asymptotic power-law decay** [11–13]. Relative to the marginal floor (1),

$$\lim_{t \to \infty} \frac{\log(\hat{L}(\mathbf{h})\big|_{x=t} - L_x^\infty)}{\log t} \;=\; c_x \qquad \text{for some } c_x < 0. \tag{2}$$

The excess $\hat{L}(\mathbf{h})|_{x=t} - L_x^\infty$ is eventually positive (so the logarithm is defined) whenever the limit in (A3)(ii) is approached from above, which follows from (A1) when the floor is not attained at finite t.

(A5) **Graceful saturation** [14]. $\hat{L}$ is twice continuously differentiable (C^2) in x on $[x_{\min}, \infty)$, so that the second derivative in (A2) exists and is continuous, and $\hat{L}(x_{\min}, \cdot)$ is finite. (Expression-tree corrections used later are in fact C^∞ on $(0, \infty)$. The inequality $\hat{L}(x_{\min}, \cdot) > L_\infty$ is already required by (A3)(i).)

These encode physical priors on loss surfaces that unconstrained symbolic regression routinely violates. In particular, (A4) must use L_x^∞ rather than L_∞: along a single axis the excess over the joint floor need not vanish, so a power-law rate would not be identifiable against L_∞.

Where the axioms come from. (A1)–(A4) are not new modeling choices; each is the formal statement of an empirical regularity already reported in the neural scaling-law literature, and we adopt them from that literature rather than positing them. Power-law scaling of generalization error in model and data size is the central finding of Hestness et al. [11] and Kaplan et al. [12], and is what (A4) asserts; the monotone decrease (A1) and the diminishing returns (A2) are the shape that a decaying power law has. Hestness et al. [11] further decompose measured learning curves into a small-data region, a power-law region, and an *irreducible-error region* bounded below by Bayes error, which is the floor L_∞ of (A3). Hoffmann et al. [13] make that floor explicit in the parametric form $\hat{L}(N, D) = E + AN^{-\alpha} + BD^{-\beta}$, in which E is read as the loss of an ideal generative process on the data distribution, i.e. the entropy of natural text. Our stage-0 law (3) is exactly that form generalized to additional hyperparameter axes, so the constants $L_\infty = E$ and $c_N^{(0)} = -\alpha$, $c_D^{(0)} = -\beta$ of Proposition 1 inherit their published meaning directly, which is what makes preserving them across boosting stages (Theorem 1) worth proving. Related joint (N, D) functional forms fitted for extrapolation appear in Rosenfeld et al. [15] and Alabdulmohsin et al. [16]. By contrast, (A5) is a regularity condition rather

than an empirical claim: it is the smoothness that derivative-based shape constraints need in order to be stated at all [14].

Scope of (A1)–(A2). Monotonicity and convexity are a modeling restriction, not a universal law. Caballero et al. [17] document *non-monotonic* scaling behavior (double descent, sharp inflections) that no monotone form can express. We impose (A1)–(A2) because this project targets the smooth, monotonically decreasing regime of LLM pretraining validation loss on the grids of Section 3.5. A testbed that exhibited breaks or double descent would require the axiom set to be relaxed; detecting such a failure is itself a legitimate outcome here, since the soft scores of Section 4.5 *measure* the admissibility gap rather than assume it away.

4.3 Boosted construction

Code skeleton: `src/search/baselines/`, `src/search/residuals/`, `src/search/expression/`

The ensemble is built stagewise, $\hat{L}^{(j)} = \hat{L}^{(j-1)} + \hat{L}_j$, targeting an *admissibility invariant*: $\hat{L}^{(j)} \in \mathcal{S}$ for every $j = 0, \ldots, K$ (Section 4.2), enforced by the shape-constrained search of Section 4.5 whenever $V = 0$ with finite DualInterval enclosures (Proposition 3 and Theorem 1). Throughout, $\hat{L}_j$ (subscript) is the stage-j correction and $\hat{L}^{(j)}$ (superscript) is the cumulative ensemble through stage j, so in particular $\hat{L}^{(0)} = \hat{L}_0$. The stagewise-additive construction is gradient boosting in the sense of Friedman [18], with the shape-constrained search of Section 4.5 in place of an unconstrained base learner. Stage 0 is a generalized Chinchilla law [13], fit by nonlinear least squares with positive parameter constraints. Concrete instances include a 2D law in (N, D) and a 4D law that adds two hyperparameter axes (e.g. weight decay and learning rate). The 4D hyperparameter terms may be power laws of the same form as in (3), or, matching U-shaped loss in lr/wd, quadratic penalties in $\log \phi_{\mathrm{lr}}(\mathrm{lr})$ and $\log \phi_{\mathrm{wd}}(\mathrm{wd})$ (shape axioms still act only on N and D):

$$\hat{L}^{(0)} \;=\; \hat{L}_0 \;=\; \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}} + \sum_{i=1}^{m} \frac{C_i}{\phi_i(h_i)^{\gamma_i}} + E, \qquad A, B, C_i, E, \alpha, \beta, \gamma_i > 0. \tag{3}$$

Proposition 1 (Stage-0 admissibility)**.** *The law (3) with all coefficients and exponents strictly positive lies in $\mathcal{S}$, with joint floor $L_\infty = E$ and asymptotic exponents $c_N^{(0)} = -\alpha$, $c_D^{(0)} = -\beta$ (the rates c_x in (A4) of Section 4.2 along N and D).*

(See Appendix C.5 for the proof.) Each later stage $j \geq 1$ fits Huber pseudo-residuals [18, 19] of the current ensemble, with clipping threshold

$$\delta_j \;\coloneqq\; \mathrm{median}_{\ell=1,\ldots,n} \big| L(\mathbf{h}_\ell) - \hat{L}^{(j-1)}(\mathbf{h}_\ell) \big| \tag{4}$$

so that outlier runs cannot dominate:

$$\tilde{r}_j^{(\ell)} \;\coloneqq\; \begin{cases} L(\mathbf{h}_\ell) - \hat{L}^{(j-1)}(\mathbf{h}_\ell) & \text{if } \big|L(\mathbf{h}_\ell) - \hat{L}^{(j-1)}(\mathbf{h}_\ell)\big| \leq \delta_j, \\ \delta_j \cdot \mathrm{sign}\big(L(\mathbf{h}_\ell) - \hat{L}^{(j-1)}(\mathbf{h}_\ell)\big) & \text{otherwise.} \end{cases} \tag{5}$$

Corrections are sought by genetic-programming search over expression trees with primary operators $\{+, -, \times, \div\} \cup \{\mathrm{pow}_p : p \in \mathcal{P}\}$ for a finite set $\mathcal{P} \subset \mathbb{R} \setminus \{0\}$ (where $\mathrm{pow}_p(z) = |z|^p$ for $z > 0$) and bounded depth; optional unary helpers such as inv/log/sqrt may be enabled for ablations. Trees are evaluated on *log-features* $\log \phi_h(h)$; raw-scale derivatives needed by (A1)–(A2) are recovered by the chain rule (Section 4.4).

4.4 Stagewise admissibility

Code skeleton: `src/constraints/axioms/stage_conditions.py`, `src/constraints/certificates/interval.py`, `src/constraints/certificates/ia_eval.py`, `src/constraints/certificates/ia_certificates.py`

Checking that the full ensemble $\hat{L}^{(j)}$ obeys (A1)–(A5) of Section 4.2 is a continuum statement on $\widetilde{\mathbb{H}}$ and along each scale half-line. The next claim reduces that check to conditions on the increment alone.

$$\begin{aligned}
&\text{(i)}\ \frac{\partial \hat{L}_j}{\partial x} \le -\frac{\partial \hat{L}^{(j-1)}}{\partial x} \quad \text{for all } x \ge x_{\min},\\
&\text{(ii)}\ \frac{\partial^2 \hat{L}_j}{\partial x^2} \ge -\frac{\partial^2 \hat{L}^{(j-1)}}{\partial x^2} \quad \text{for all } x \ge x_{\min},\\
&\text{(iii)}\ \hat{L}_j(\mathbf{h}) > -\hat{L}^{(j-1)}(\mathbf{h}) \quad \text{for all } \mathbf{h} \in \widetilde{\mathbb{H}},\\
&\text{(iv)}\ \hat{L}^{(j-1)}(\mathbf{h}) + \hat{L}_j(\mathbf{h}) > L_\infty \quad \text{for all } \mathbf{h} \in \widetilde{\mathbb{H}},\\
&\text{(v)}\ \hat{L}_j(\mathbf{h})\big|_{x=t} = O\big(t^{c_{x,j}}\big) \text{ as } t \to \infty \text{ for some } c_{x,j} < c_x^{(0)},\\
&\text{(vi)}\ \hat{L}_j \in C^\infty \text{ on } [x_{\min}, \infty) \text{ with } |\hat{L}_j(x_{\min}, \cdot)| < \infty.
\end{aligned} \tag{6}$$

When $L_\infty > 0$, (iv) already implies (iii); both are retained so positivity of the ensemble and preservation of the joint floor remain explicit. Note that (vi) is required on the whole half-line, not merely on the certification box: the (A5) half of Proposition 2 needs $\hat{L}^{(j-1)} + \hat{L}_j$ to stay C^2 wherever (A1)–(A2) are asserted. For a finite expression tree this amounts to no division denominator and no pow_p argument vanishing on $[x_{\min}, \infty)$; interval arithmetic discharges it on $\mathcal{I}_x$ and the remainder is a structural side condition, as discussed in (a) below.

Proposition 2 (Stagewise conditions)**.** *Let $\hat{L}^{(j-1)} \in \mathcal{S}$ have joint floor L_∞ and stage-0 rates $c_x^{(0)}$. If $\hat{L}_j$ satisfies (6)(i)–(vi) for each $x \in \{N, D\}$, then $\hat{L}^{(j)} = \hat{L}^{(j-1)} + \hat{L}_j$ lies in $\mathcal{S}$ with the same joint floor and the same asymptotic rates. Conversely, any update with those three properties satisfies (i)–(iv) and (vi) together with the tail bound $\hat{L}_j(\mathbf{h})|_{x=t} = o\big(t^{c_x^{(0)}}\big)$. The gap between that bound and (v) is deliberate: (v) is the strictly stronger form, since it excludes borderline tails such as $t^{c_x^{(0)}}/\log t$, and we adopt it because it is what the structural test of (b) decides exactly on a finite tree, while only the sufficient direction is needed by Theorem 1.*

(See Appendix C.1 for the proof.) Conditions (i)–(iv) and (vi) are still quantified over continua ((i)–(ii) on the half-line $x \ge x_{\min}$; (iii)–(iv) on $\widetilde{\mathbb{H}}$), and (v) is an asymptotic statement: none of these can be decided by sampling finitely many points of $\mathbb{H}$. They become machine-checkable because every GP proposal g is a concrete finite expression tree built from $\{+, -, \times, \div\} \cup \{\mathrm{pow}_p : p \in \mathcal{P}\}$. Write $F \coloneqq \hat{L}^{(j-1)} + g$. Two complementary certificate constructions supply the continuous checks used by the soft fitness of Section 4.5: interval enclosures establish (i)–(iv) and (vi) on the compact box $\mathcal{I}_x$ covering the observed grid, while the structural check decides (v) exactly (and thereby controls the $x \to \infty$ tail that the box does not cover).

(a) Sound interval-arithmetic checks (conditions (i)–(iv), (vi) on a compact slice of $\widetilde{\mathbb{H}}$). Over the compact box $\mathcal{I}_x \coloneqq [x_{\min}, x_{\max}]$ (other coordinates ranging over their finite H_h), the product of boxes is a compact continuum subset of $\widetilde{\mathbb{H}}$ covering the observed scale range; the half-line $x > x_{\max}$ is handled by the structural test (b) below. Forward-mode automatic differentiation composed with interval arithmetic [14, 20] evaluates g bottom-up on interval operands and returns guaranteed enclosures $\underline{f} \le f \le \overline{f}$ on $\mathcal{I}_x$ for $f \in \{F, \partial F/\partial x, \partial^2 F/\partial x^2\}$, in time linear in the tree size. (The second-derivative enclosure is what makes (A2) / stage (ii) checkable; (A5) asks only for C^2, while the operator set of Section 4.3 in fact yields C^∞ on $(0, \infty)$.) When g is represented on log-features, DualInterval evaluation

is performed in log-coordinates and first/second derivatives are mapped to the raw axis x by the chain rule before applying (7). Those enclosures are exactly the inputs to the soft violation scores of Section 4.5: the per-axis monotonicity gap is $\max(0, \overline{\partial F/\partial x})$ and likewise for the other gaps, with v_{mono} and its companions in (8) taking $\max_{x\in\{N,D\}}$ of those per-axis quantities. The sufficient inequalities

$$\overline{\partial F/\partial x} \le 0, \quad \underline{\partial^2 F/\partial x^2} \ge 0, \quad \underline{F} > L_\infty, \quad \overline{F} < \infty \tag{7}$$

hold for each $x \in \{N, D\}$ precisely when the corresponding soft gaps vanish (and $\overline{F} < \infty$). Because the enclosures are sound, vanishing gaps are a *sufficient* certificate that the continuum conditions hold *everywhere* on $\mathcal{I}_x$, not merely at sampled points of $\mathbb{H}$ (Lemma 1 and Appendix C.2): $\overline{\partial F/\partial x} \le 0$ forces $\partial F/\partial x \le 0$ on $\mathcal{I}_x$ (hence (i) on the box), and likewise for (ii); $\underline{F} > L_\infty > 0$ forces (iv) on the continuum configurations covered by the product of boxes, and hence also (iii) (ensemble positivity); $\overline{F} < \infty$ forces the finiteness half of (vi), and, because a finite enclosure also certifies that no denominator or pow_p argument vanished, yields C^2 (in fact C^∞) smoothness of g on $\mathcal{I}_x$, discharging (A5) on the box. The implication is one-sided: inflated enclosures can produce positive soft scores for a truly admissible update, but every g with vanishing gaps on (7) is guaranteed to preserve admissibility on $\mathcal{I}_x$. Evaluating DualInterval enclosures (and the soft scores built from them) on each GP candidate is the inner-loop hot spot, the direct target of the AMA27 performance work.

Where the smoothness in (A5) actually comes from. The primitives are smooth only away from the zeros of their arguments: z_1/z_2 is C^∞ where $z_2 \neq 0$, and $\text{pow}_p(z) = |z|^p$ is C^∞ where $z \neq 0$ (at the origin it is unbounded for $p < 0$ and, for the fractional p that dominate a useful $\mathcal{P}$, not even C^2). A finite tree g is therefore C^∞ precisely on the set where every division denominator and every pow_p argument is nonzero; the shorthand "the operator set is C^∞ on $(0, \infty)$" used below abbreviates that condition and is *not* a consequence of the tree merely being finite. On the box the condition is discharged for free *provided the DualInterval layer is implemented to fail loudly*: an operand interval straddling 0 beneath $\div$ or pow_p must yield a non-finite enclosure, so a completed evaluation with $\overline{F} < \infty$ certifies that no such subtree vanished anywhere on $\mathcal{I}_x$, and with it (A5) and the finiteness half of (vi). Beyond $x_{\max}$ there is no enclosure to lean on, so the tail half of (A5) carries the side condition that no denominator or pow_p argument vanishes on $(x_{\max}, \infty)$ either; an implementation discharges it by restricting divisors to subtrees of certified sign, or by widening $\mathcal{I}_x$ past the range over which extrapolations are actually reported.

(b) Exact structural checks (condition (v), and prerequisites for it). Define the asymptotic exponent $\text{ord}(g, x)$ recursively on the tree: constants and leaves other than x contribute 0; $\text{pow}_p(x)$ contributes p; products add exponents; quotients subtract; sums take the maximum. Then (v) holds with $c_{x,j} = \text{ord}(g, x)$ whenever $\text{ord}(g, x) < c_x^{(0)}$ for each $x \in \{N, D\}$ (Lemma 2 and Appendix C.2); when two sibling subtrees share the leading exponent their leading coefficients may cancel, in which case ord is a sound over-estimate of the true order and the test stays sufficient. We additionally require that each scale variable $x \in \{N, D\}$ appears as a leaf of g; this is a prerequisite for the exponent rather than a constraint in its own right, since otherwise $\text{ord}(g, x)$ is vacuously 0 and (v) could be passed by a correction that ignores that axis. Both tests are exact, finite walks over a finite tree, with no floating-point search over $\widetilde{\mathbb{H}}$.

4.5 Shape-constrained search

Code skeleton: `src/search/soft/violations.py`, `src/search/soft/fitness.py`, `src/search/soft/backends.py`

The discovery method, and the only search used by Algorithm 1, is soft interval-arithmetic search: per-axiom violation scores on the candidate ensemble $F := \hat{L}^{(j-1)} + g$ are folded into the `gplearn` fitness (Section 3.3), steering the genetic program toward admissible corrections. (The student

package is `src/search/soft/`; there is no separate reject-filter path.) Using the interval enclosures of Section 4.4(a) and the structural quantities of (b), define the soft scores of Appendix A:

$$\begin{aligned}
v_{\text{mono}}(g) &= \max_{x\in\{N,D\}} \max\Big(0,\ \overline{\partial F/\partial x}\Big), && \text{(monotonicity gap)},\\
v_{\text{conv}}(g) &= \max_{x\in\{N,D\}} \max\Big(0,\ \underline{-\partial^2 F/\partial x^2}\Big), && \text{(convexity gap)},\\
v_{\text{irred}}(g) &= \max_{x\in\{N,D\}} \max(0,\ L_\infty - \underline{F}), && \text{(floor gap)},\\
v_{\text{decay}}(g) &= \max_{x\in\{N,D\}} \max(0,\ \text{ord}(g,x) - c_x^{(0)} + \varepsilon_{\text{decay}}), && \text{(power-law gap)},\\
v_{\text{leaf}}(g) &= \big|\{x\in\{N,D\} : x \notin \text{leaves}(g)\}\big|, && \text{(missing scale leaves)}.
\end{aligned} \tag{8}$$

Here $\varepsilon_{\text{decay}} > 0$ is a fixed margin so that $v_{\text{decay}}(g) = 0$ forces the strict inequality $\text{ord}(g,x) \le c_x^{(0)} - \varepsilon_{\text{decay}} < c_x^{(0)}$ required by (6)(v); leaves(g) is the set of variable leaves of the expression tree g. Every score is nonnegative, and $v_{\text{mono}} = v_{\text{conv}} = v_{\text{irred}} = v_{\text{leaf}} = 0$ iff the corresponding interval or structural test of (7) / (b) succeeds (for v_{decay}, zero is sufficient but slightly stronger than (v) because of the margin). Writing a for an axiom index ranging over $\{\text{mono}, \text{conv}, \text{irred}, \text{decay}, \text{leaf}\}$ and $\lambda_a > 0$ for its penalty weight, the aggregate violation is $V(g) = \sum_a v_a(g)$. Stage j then minimizes the penalized fitness

$$F_j(g) \;=\; \underbrace{\frac{1}{n}\sum_{\ell=1}^{n}\big(\tilde r_j^{(\ell)} - g(\mathbf{h}_\ell)\big)^2}_{\text{data fit}} \;+\; \underbrace{\sum_a \lambda_a\, v_a(g)^2}_{\text{axiom penalty}}, \tag{9}$$

where F_j is a scalar fitness (distinct from the ensemble map F above), the first term is mean squared error of g against the Huber pseudo-residuals (5), and the second term penalizes axiom gaps (squared so the penalty gradient vanishes at zero violation). Candidates with vanishing soft scores (and finite DualInterval enclosures) therefore incur no penalty pressure; note that $v_{\text{decay}} = 0$ is slightly stronger than (6)(v) because of the margin $\varepsilon_{\text{decay}}$.

Where the penalties come from. Constraining a symbolic-regression model through bounds on its image and its partial derivatives, and computing those bounds with interval arithmetic, is the shape-constrained symbolic regression of Kronberger et al. [14]; Błądek and Krawiec [21] attack the same problem with SMT solvers and counterexample-driven GP instead. Our v_{mono}, v_{conv}, and v_{irred} are that literature's monotonicity, convexity, and image-bound constraints, specialized to (A1)–(A3) of Section 4.2 and evaluated on the ensemble $F = \hat L^{(j-1)} + g$ rather than on the candidate g alone, the modification that makes them meaningful stagewise (Proposition 2). Where we depart from Kronberger et al. [14] is the *handling* of a violation: they enforce constraints as a hard feasibility filter, discarding infeasible candidates during selection or segregating them into a second population, whereas (9) adds $\sum_a \lambda_a v_a(g)^2$ to the fitness and lets violations shrink continuously. That soft, penalty-based treatment follows Martinek et al. [22], who minimize shape-constraint violations inside the objective rather than applying them only as a post-hoc selection filter, and who report the clearest gains in the data-scarce regime, which is the regime a scaling grid of expensive pretraining runs puts us in. The one-sidedness recorded after (7) (inflated enclosures may penalize a candidate that is in fact admissible, but never certify one that is not) is the *pessimistic* constraint evaluation compared against optimistic sampling-based evaluation by Haider et al. [23]; we take the pessimistic side deliberately, because Theorem 1 needs soundness rather than tightness. The remaining two scores have no counterpart in that literature: v_{decay} encodes the asymptotic-rate axiom (A4), which is specific to scaling laws, through the structural exponent $\text{ord}(g,x)$ of Section 4.4(b) on the half-line $x > x_{\max}$ rather than through an interval bound on a compact box, and v_{leaf} is the prerequisite that makes that exponent meaningful.

Proposition 3 (Zero-penalty implies certificates)**.** *If $V(g) = 0$ at every stage* and *DualInterval evaluation returns finite upper enclosures $\overline{F} < \infty$ on each $\mathcal{I}_x$ (the finiteness half of (7), discharged in practice by the C^∞ operator set of Section 4.3), then certificates (a)–(b) hold and the boosting guarantee of Theorem 1 applies exactly. The scores v_{mono}, v_{conv}, v_{irred} upper-bound the corresponding true pointwise violations on $\mathcal{I}_x$ (while v_{decay} and v_{leaf} are exact).*

(See Appendix C.3 for the proof.) The soft scores therefore yield a *measured* admissibility gap, in contrast to unconstrained search ($\lambda_a = 0$), which can fit the grid while leaving axiom violations unpenalized. Soft scores act as IA proxies for (A1)–(A4) (with (A5) discharged by finiteness of the DualInterval enclosure / C^2 (in practice C^∞) operator set). Comparing soft-penalty search ($\lambda_a > 0$) against that unconstrained baseline is one of the empirical questions the project settles.

4.6 Algorithm

Code skeleton: `src/search/boosting/algorithm.py`, `src/search/boosting/`

Putting the pieces together, Algorithm 1 builds the ensemble of Section 4.3. At every stage the correction is the minimizer of the shape-constrained fitness (9) from Section 4.5.

Algorithm 1 Shape-constrained boosted symbolic regression

Require: Dataset $\mathcal{D} = \{(\mathbf{h}_\ell, L(\mathbf{h}_\ell))\}_{\ell=1}^n$, stages K
Ensure: Ensemble $\hat{L}^{(K)}$ (aims for $V = 0$ with finite DualInterval enclosures at every stage)

```
Fit L̂_0 via (3) on D; set L̂^(0) ← L̂_0
for j = 1, ..., K do
    Compute δ_j via (4)
    Compute r̃_j^(ℓ) for all ℓ = 1, ..., n via (5)
    Find L̂_j by minimizing (9)
    L̂^(j) ← L̂^(j−1) + L̂_j
end for
return L̂^(K)
```

4.7 Guarantee

Code skeleton: `src/constraints/guarantee/invariants.py`

The soft search of Section 4.5 drives each correction toward vanishing violation scores. When $V(\hat{L}_j) = 0$ with finite DualInterval enclosures at every stage (Proposition 3), certificates (a)–(b) of Section 4.4 hold, and the ensemble preserves admissibility, the joint floor, and the stage-0 rates:

Theorem 1 (Boosting guarantee)**.** *Assume $\hat{L}_0$ is the admissible stage-0 law of Proposition 1 (so $L_\infty = E$). If every later correction $\hat{L}_j$ ($j \geq 1$) has $V(\hat{L}_j) = 0$ with finite DualInterval enclosures (equivalently, satisfies certificates (a)–(b) of Section 4.4; see Proposition 3), then for all j: $\hat{L}^{(j)} \in \mathcal{S}$; writing $L_\infty^{(j)}$ for the joint floor of $\hat{L}^{(j)}$, one has $L_\infty^{(j)} = E$; and the asymptotic exponents are preserved,*

$$\lim_{t\to\infty} \frac{\log(\hat{L}^{(j)}(\mathbf{h})|_{x=t} - L_x^\infty)}{\log t} = c_x^{(0)}, \tag{10}$$

where L_x^∞ is the marginal floor of $\hat{L}^{(j)}$ along x (equal to that of $\hat{L}_0$).

Boosting therefore improves fit *without* corrupting the interpretable constants researchers actually quote (proofs in Appendices C.3 and C.4).

5 Additional information

5.1 Collaborators welcome

Code skeleton: `src/modeling/` (modeling); `src/search/`, `src/systems/hpc/` (search)

The project splits into two fairly independent tracks, so a teammate could own either one. The *modeling* track ports and benchmarks transformer pretraining on Neocortex's Cerebras CS-3 systems (Section 3.1) and compares the resulting validation losses to the public Hugging Face loss curves of Section 3.2 (useful background: PyTorch, or curiosity about the Cerebras software stack). The *search* track ports the genetic-programming symbolic-regression loop of Sections 4.3, 4.5 and 4.6 to AMA27 (Section 3.1) and profiles the DualInterval certification inner loop of Section 4.4 on AmpereOne (useful background: C/C++ or Python performance work, or an interest in Arm/HPC profiling); a self-contained slice of it is benchmarking soft-penalty search against $\lambda_a = 0$ (Section 4.5). No prior scaling-law expertise is needed for either; the main thing is an appetite for making code run fast on unusual hardware. The student skeleton paths for each track are listed in Appendix B.

5.2 Deliverables

Code skeleton: `src/systems/pipeline/run_discovery.py`

(1) An admissible boosted scaling law (Sections 4.2 and 4.7) fit on the collected training grid (Section 3.5); (2) a public release of the shape-constrained symbolic-regression library with the DualInterval soft-score / certification stack of Sections 4.4 and 4.5; and (3) a short paper on the method of Section 4. Notation, the description↔skeleton map, and the formal proofs appear in Appendices A to C.

6 References

[1] Leibniz Lab. colinear_scaling_models: Scaling-grid checkpoints and loss curves. https://huggingface.co/datasets/leibnitz-lab/colinear_scaling_models, 2025. Hugging Face dataset.

[2] Lukas Biewald. Experiment tracking with weights & biases. https://wandb.ai/, 2020. Software available from wandb.com.

[3] Trevor Stephens. gplearn: Genetic programming in Python. https://gplearn.readthedocs.io/, 2015. Software library.

[4] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and efficient foundation language models, 2023. URL https://arxiv.org/abs/2302.13971.

[5] Greg Yang, Edward J. Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tensor programs v: Tuning large neural networks via zero-shot hyperparameter transfer. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. URL https://arxiv.org/abs/2203.03466. arXiv:2203.03466.

[6] Houyi Li, Wenzhen Zheng, Qiufeng Wang, Hanshan Zhang, Zili Wang, Shijie Xuyang, et al. Predictable scale: Part i, step law: Optimal hyperparameter scaling law in large language model pretraining, 2025. URL https://arxiv.org/abs/2503.04715.

[7] Samir Yitzhak Gadre, Georgios Smyrnis, Vaishaal Shankar, Suchin Gururangan, Mitchell Wortsman, Rulin Shao, Jean Mercat, et al. Language models scale reliably with over-training and on downstream tasks, 2024. URL https://arxiv.org/abs/2403.08540.

[8] Wikimedia Foundation. Wikimedia downloads. https://dumps.wikimedia.org, 2023. Wikipedia dumps; also available as https://huggingface.co/datasets/wikimedia/wikipedia.

[9] Together Computer. RedPajama: An open dataset for training large language models. https://huggingface.co/datasets/togethercomputer/RedPajama-Data-1T, 2023. Hugging Face dataset.

[10] Guilherme Penedo et al. FineWeb-Edu: Educational subset of FineWeb. https://huggingface.co/datasets/HuggingFaceFW/fineweb-edu, 2024. Hugging Face dataset.

[11] Joel Hestness, Sharan Narang, Newsha Ardalani, Gregory Diamos, Heewoo Jun, Hassan Kianinejad, Md. Mostofa Ali Patwary, Yang Yang, and Yanqi Zhou. Deep learning scaling is predictable, empirically, 2017. URL https://arxiv.org/abs/1712.00409.

[12] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020. URL https://arxiv.org/abs/2001.08361.

[13] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models, 2022. URL https://arxiv.org/abs/2203.15556. Advances in Neural Information Processing Systems (NeurIPS) 35.

[14] G. Kronberger, F. O. de França, B. Burlacu, C. Haider, and M. Kommenda. Shape-constrained symbolic regression: Improving extrapolation with prior knowledge. *Evolutionary Computation*, 30 (1):75–98, 2022. doi: 10.1162/evco_a_00294. URL https://doi.org/10.1162/evco_a_00294.

[15] Jonathan S. Rosenfeld, Amir Rosenfeld, Yonatan Belinkov, and Nir Shavit. A constructive prediction of the generalization error across scales, 2020. URL https://arxiv.org/abs/1909.12673. International Conference on Learning Representations (ICLR).

[16] Ibrahim Alabdulmohsin, Behnam Neyshabur, and Xiaohua Zhai. Revisiting neural scaling laws in language and vision, 2022. URL https://arxiv.org/abs/2209.06640. Advances in Neural Information Processing Systems (NeurIPS) 35.

[17] Ethan Caballero, Kshitij Gupta, Irina Rish, and David Krueger. Broken neural scaling laws, 2023. URL https://arxiv.org/abs/2210.14891. International Conference on Learning Representations (ICLR).

[18] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451. URL https://doi.org/10.1214/aos/1013203451.

[19] Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964. doi: 10.1214/aoms/1177703732. URL https://doi.org/10.1214/aoms/1177703732.

[20] Ramon E. Moore, R. Baker Kearfott, and Michael J. Cloud. *Introduction to Interval Analysis*. Society for Industrial and Applied Mathematics (SIAM), 2009. doi: 10.1137/1.9780898717716. URL https://doi.org/10.1137/1.9780898717716.

[21] Iwo Błądek and Krzysztof Krawiec. Solving symbolic regression problems with formal constraints. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '19)*, pages 977–984. ACM, 2019. doi: 10.1145/3321707.3321743. URL https://doi.org/10.1145/3321707.3321743.

[22] Viktor Martinek, Julia Reuter, Ophelia Frotscher, Sanaz Mostaghim, Markus Richter, and Roland Herzog. Shape constraints in symbolic regression using penalized least squares. In *Machine Learning and Principles and Practice of Knowledge Discovery in Databases (ECML PKDD 2024 Workshops)*, Communications in Computer and Information Science, pages 224–239. Springer, 2024. doi: 10.1007/978-3-032-25305-7_16. URL https://arxiv.org/abs/2405.20800. arXiv:2405.20800.

[23] Christian Haider, Fabrício Olivetti de França, Gabriel Kronberger, and Bogdan Burlacu. Comparing optimistic and pessimistic constraint evaluation in shape-constrained symbolic regression. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '22)*, pages 938–945. ACM, 2022. doi: 10.1145/3512290.3528714. URL https://doi.org/10.1145/3512290.3528714.

A Notation

Symbols used in Section 4 (and pointed to from the student stubs of Appendix B). Formal claims from Section 4 are proved in Appendix C. Violation scores v_a, aggregate V, and fitness F_j belong to the sole discovery method of Section 4.5.

Symbol	Meaning
N, D	Parameter count and training-token count (scale variables).
M	Tokens-per-parameter ratio $M = D/N$ (Section 3.4).
$\mathcal{H} = \{h_1, \ldots, h_m\}$	Remaining hyperparameters; $m = \lvert\mathcal{H}\rvert$.
i	Hyperparameter index, $i = 1, \ldots, m$ (distinct from data-point index ℓ).
H_h	Finite set of admissible values for coordinate $h \in \{N, D\} \cup \mathcal{H}$.
$\phi_h,\ \phi_i$	Strictly monotone preprocessing $\phi_h \colon H_h \to \mathbb{R}_{>0}$; $\phi_i \coloneqq \phi_{h_i}$.
$\mathbb{H}$	Discrete configuration space $\prod_h H_h$ (labeled grid).
$\widetilde{\mathbb{H}}$	Continuum domain of Sections 4.1 and 4.2: each scale ranges over $[x_{\min}, \infty)$, other coordinates over finite H_h; $\mathbb{H} \subset \widetilde{\mathbb{H}}$.
$\mathbf{h}$	Typical element of $\mathbb{H}$ or $\widetilde{\mathbb{H}}$ (coordinates N, D, h_i).
$L,\ \hat{L}$	True validation loss and its approximation.
$\mathcal{D},\ n,\ \ell$	Labeled dataset $\{(\mathbf{h}_\ell, L(\mathbf{h}_\ell))\}_{\ell=1}^{n}$; $n = \lvert\mathcal{D}\rvert$; data-point index ℓ.
x	Generic scale variable, $x \in \{N, D\}$.
t	Dummy limit variable along a scale axis (e.g. $\hat{L}(\mathbf{h})\lvert_{x=t}$).
$x_{\min},\ x_{\max}$	$\min H_x$ and $\max H_x$.
$\mathcal{S}$	Set of admissible scaling laws (Section 4.2).
$L_\infty,\ L_\infty^{(j)}$	Joint irreducible-loss floor; joint floor of $\hat{L}^{(j)}$ ($L_\infty^{(j)} = E$ when $V = 0$ with finite enclosures).
L_x^∞	Marginal floor along axis x (1) (depends on the frozen coordinates).
$c_x,\ c_x^{(0)}$	Asymptotic power-law exponent along x (A4); stage-0 value $c_N^{(0)} = -\alpha$, $c_D^{(0)} = -\beta$.
$j,\ K$	Boosting stage index; number of correction stages after stage 0 (Section 4.6, Algorithm 1).
$\hat{L}_j,\ \hat{L}^{(j)}$	Stage-j correction (subscript) and cumulative ensemble through stage j (superscript); $\hat{L}^{(0)} = \hat{L}_0$.
$A, B, C_i, E, \alpha, \beta, \gamma_i$	Positive coefficients/exponents of the stage-0 Chinchilla law (3); joint floor $L_\infty = E$.
δ_j	Huber clipping threshold at stage j (4).
$\tilde{r}_j^{(\ell)}$	Huber pseudo-residual for data point ℓ at stage j (5).
$\mathcal{P},\ p$	Finite set of admissible powers $\mathcal{P} \subset \mathbb{R} \setminus \{0\}$; generic element p.
pow_p	Power primitive $\mathrm{pow}_p(z) = \lvert z\rvert^p$ for $z > 0$.
$g,\ F$	Candidate expression tree; ensemble-plus-candidate $F = \hat{L}^{(j-1)} + g$.
$c_{x,j},\ \mathrm{ord}(g, x)$	Asymptotic exponent of correction/candidate g along x; $c_{x,j} = \mathrm{ord}(g, x)$ in (6)(v).
$\mathcal{I}_x$	Compact certification box $[x_{\min}, x_{\max}]$ along scale x (written I_x in the skeleton); with other coordinates over finite H_h, the product of boxes is a compact continuum slice of $\widetilde{\mathbb{H}}$ covering the observed grid (Section 4.4); the tail $x > x_{\max}$ is controlled by ord.

Symbol	Meaning
$\underline{f}$, $\overline{f}$	Interval-arithmetic lower/upper enclosures of f on $\mathcal{I}_x$ (global over that box for fixed x; soft scores take $\max_{x\in\{N,D\}}$ of the per-axis gaps). $\underline{F} > L_\infty$ certifies (A3) on the box (hence ensemble positivity); $\overline{F} < \infty$ certifies (A5) on the box, since a completed finite evaluation also rules out a vanishing denominator or pow_p argument (Section 4.4(a)).
a, $v_a(g)$, $V(g)$	Soft-search axiom index $a \in \{\mathrm{mono}, \mathrm{conv}, \mathrm{irred}, \mathrm{decay}, \mathrm{leaf}\}$; per-axiom violation score; aggregate $V = \sum_a v_a$ (8).
$\varepsilon_{\mathrm{decay}}$	Positive margin in v_{decay} so zero score implies strict power-law decay (8).
$\mathrm{leaves}(g)$	Set of variable leaves of expression tree g.
λ_a	Positive penalty weight for axiom a.
$F_j(g)$	Shape-constrained penalized fitness at stage j (9) (distinct from F).

B Code skeleton map

The repository ships a student implementation skeleton under `src/`, grouped into five packages (`src/README.md`). Blue links open the corresponding paths on GitHub (`main`). Each technical subsection in the main text also points to those files via the *Code skeleton* lines; symbols used there are listed in Appendix A. The stage-0 baseline used by the boosting stubs is Proposition 1; proofs of all formal claims are in Appendix C. The table follows the soft-only method order of Section 4: DualInterval enclosures feed the sole discovery package `src/search/soft/` (Section 4.5), which Algorithm 1 calls at every stage; there is no hard reject-filter package.

Description	Package	Primary paths
Section 4.1	`src/scaling/`	`src/scaling/setup/configuration.py`, `src/scaling/setup/constants.py`
Section 3.5	`src/scaling/`	`src/scaling/data/scaling_dataset.py`, `src/scaling/data/corpora.py`
Section 4.2	`src/constraints/`	`src/constraints/axioms/admissibility.py`
Section 4.3	`src/search/`	`src/search/baselines/`, `src/search/residuals/`, `src/search/expression/`
Section 4.4	`src/constraints/`	`src/constraints/axioms/stage_conditions.py`, `src/constraints/certificates/interval.py`, `src/constraints/certificates/ia_eval.py`, `src/constraints/certificates/ia_certificates.py`
Section 4.5	`src/search/`	`src/search/soft/violations.py`, `src/search/soft/fitness.py`, `src/search/soft/backends.py`
Section 4.6, Alg. 1	`src/search/`	`src/search/boosting/algorithm.py`, `src/search/boosting/`
Section 4.7	`src/constraints/`	`src/constraints/guarantee/invariants.py`
Section 3.3 (optional JAX)	`src/constraints/`	`src/constraints/certificates/jax_backend.py`
Section 3.2	`src/scaling/`	`src/scaling/data/scaling_dataset.py`

Description	Package	Primary paths
Section 3.4	`src/modeling/`	`src/modeling/models/config.py`, `src/modeling/models/transformer.py`
Section 3.1 (Neocortex)	`src/modeling/`	`src/modeling/training/`
Section 3.1 (AMA27/CPU)	`src/systems/`	`src/systems/hpc/`
Section 5.2	`src/systems/`	`src/systems/pipeline/run_discovery.py`

C Proofs

Proofs follow the dependency order of Section 4: the stagewise conditions, then the certificate lemmas, then zero-penalty implies certificates, then the boosting guarantee. Stage-0 admissibility (Proposition 1) is independent of the boosting loop and is proved last.

C.1 Stagewise conditions

Proof of Proposition 2. Write $G := \hat{L}^{(j-1)}$ and $g := \hat{L}_j$, so $\hat{L}^{(j)} = G + g$. Throughout, derivatives along $x \in \{N, D\}$ hold the other coordinates fixed.

(A1)–(A2). $\partial(G+g)/\partial x = \partial G/\partial x + \partial g/\partial x$ and likewise for second derivatives. Since $G \in \mathcal{S}$, one has $\partial G/\partial x \le 0$ and $\partial^2 G/\partial x^2 \ge 0$ on $[x_{\min}, \infty)$. Thus $\partial(G+g)/\partial x \le 0$ on that half-line if and only if (i) holds, and $\partial^2(G+g)/\partial x^2 \ge 0$ if and only if (ii) holds.

Joint floor (A3) and ensemble positivity. The joint-floor inequality $G + g > L_\infty$ on $\widetilde{\mathbb{H}}$ is exactly (iv). (Here L_∞ is the joint floor already certified for G; (iv) keeps it for the update. When $L_\infty > 0$, (iv) also yields ensemble positivity $G + g > 0$, which is (iii).)

(A5). If G is C^2 on $[x_{\min}, \infty)$ and g is C^∞ on $[x_{\min}, \infty)$ with finite boundary values (vi), then $G + g$ is C^2 on $[x_{\min}, \infty)$, which is (A5). The two halves of that hypothesis on g are certified separately: on the box $\mathcal{I}_x = [x_{\min}, x_{\max}]$ by the interval layer, and beyond it structurally. On the half-line beyond $x_{\max}$, the same C^2 regularity follows once g is realized by a finite expression tree from $\{+, -, \times, \div\} \cup \{\mathrm{pow}_p\}$ *and* no division denominator or pow_p argument vanishes there, since each primitive is C^∞ away from the zeros of its arguments (Section 4.4(a), "where the smoothness in (A5) actually comes from"); that nonvanishing side condition is what the tail half of (vi) asks for, and it is the setting of Section 4.3. Finiteness of $(G+g)(x_{\min}, \cdot)$ is the restriction of (vi) together with $G \in \mathcal{S}$; the bound $(G+g)(x_{\min}, \cdot) > L_\infty$ is the restriction of (iv).

(A3) marginal floors and (A4). Because $G \in \mathcal{S}$, the marginal floor $L_x^\infty(G)$ exists and, by (A4), the excess satisfies $G|_{x=t} - L_x^\infty(G) = t^{\,c_x^{(0)} + o(1)}$ with $c_x^{(0)} < 0$. (We use only this log-rate form, not the stronger "constant times $t^{c_x^{(0)}}$", which (A4) does not assert.) Condition (v) gives $g|_{x=t} = O(t^{c_{x,j}})$ with $c_{x,j} < c_x^{(0)}$; since $t^{c_{x,j}} / t^{\,c_x^{(0)} + o(1)} = t^{\,c_{x,j} - c_x^{(0)} + o(1)} \to 0$, the correction is negligible against the excess, and in particular $g|_{x=t} = o\big(t^{c_x^{(0)}}\big)$. Therefore

$$\lim_{t \to \infty} (G+g)\big|_{x=t} \;=\; L_x^\infty(G),$$

the marginal floor is unchanged, and

$$\lim_{t\to\infty} \frac{\log((G+g)|_{x=t} - L_x^{\infty}(G))}{\log t} \;=\; c_x^{(0)}.$$

Converse. Suppose $G+g \in \mathcal{S}$ with the same joint floor and the same asymptotic rate $c_x^{(0)}$. Items (i)–(iv) and (vi) are the rearrangements already used above, read in the opposite direction. For the tail, the difference $g = (G+g) - G$ must be $o(t^{c_x^{(0)}})$ along each axis, since otherwise the leading excess exponent of $G+g$ would differ from that of G. This is weaker than (v): $o(t^{c_x^{(0)}})$ does not supply any single exponent $c_{x,j} < c_x^{(0)}$ with $g = O(t^{c_{x,j}})$, as $g = t^{c_x^{(0)}}/\log t$ shows. So (i)–(vi) are sufficient, and necessary only with (v) relaxed to the $o(\cdot)$ bound. We work with (v) rather than the exact necessary condition because (v) is what ord decides exactly on a finite tree (Lemma 2), and because only the sufficient direction is used in Theorem 1. □

C.2 Interval and structural certificates

Lemma 1 (Soundness of the interval tests)**.** *Suppose forward-mode DualInterval evaluation returns sound enclosures $\underline{f} \le f \le \overline{f}$ on $\mathcal{I}_x$ for $f \in \{F, \partial F/\partial x, \partial^2 F/\partial x^2\}$. If (7) holds, then on $\mathcal{I}_x$: $\partial F/\partial x \le 0$, $\partial^2 F/\partial x^2 \ge 0$, and $F > L_\infty$, and $|F| < \infty$. In particular, writing $F = G + g$ with $G = \hat{L}^{(j-1)}$, conditions (i)–(iv) and the finiteness half of (vi) hold on the continuum configurations covered by the product of boxes $\mathcal{I}_x$ (a compact slice of $\widetilde{\mathbb{H}}$). When $L_\infty > 0$, the floor test also yields (iii). Smoothness (A5) on the box follows once finiteness holds, because a completed finite evaluation certifies that no division denominator or pow_p argument vanished on $\mathcal{I}_x$, and the operator set is C^∞ there (Section 4.4(a)).*

Proof. Soundness means every true value on $\mathcal{I}_x$ lies in the returned interval. Hence $\overline{\partial F/\partial x} \le 0$ forces $\partial F/\partial x \le 0$ everywhere on $\mathcal{I}_x$; $\underline{\partial^2 F/\partial x^2} \ge 0$ forces $\partial^2 F/\partial x^2 \ge 0$; and $\underline{F} > L_\infty$ forces $F > L_\infty$ on the box (hence also $F > 0$ when $L_\infty > 0$), which is (iv) and yields (iii). Boundedness of $\overline{F}$ is the finiteness claim of (vi). The argument is one-sided: failure of (7) does not refute (6). □

Lemma 2 (Structural exponent)**.** *Let g be a finite expression tree over $\{+, -, \times, \div\} \cup \{\mathrm{pow}_p : p \in \mathcal{P}\}$ in which the scale leaf x appears. Define $\mathrm{ord}(g, x)$ recursively as in Section 4.4(b). Then $g(\mathbf{h})|_{x=t} = O(t^{\mathrm{ord}(g,x)})$ as $t \to \infty$, with matching Θ asymptotics whenever no exact cancellation of leading terms occurs. Hence (6)(v) holds with $c_{x,j} = \mathrm{ord}(g, x)$ whenever $\mathrm{ord}(g, x) < c_x^{(0)}$, and the test is exact on trees free of leading-term cancellation.*

Proof. Proceed by induction on tree size. Constants and leaves other than x are $O(1) = \Theta(t^0)$. A factor $\mathrm{pow}_p(x)$ contributes $\Theta(t^p)$. Products multiply leading powers (exponents add); quotients subtract; a sum is dominated by the child of largest exponent, so the inductive clause "sums take the maximum" records the leading order. (If two children share the same maximal exponent their leading coefficients might cancel, in which case the true order is strictly smaller; the recursive ord is then a sound upper bound, and the test $\mathrm{ord}(g, x) < c_x^{(0)}$ remains sufficient for (v).) Thus $g|_{x=t} = O(t^{\mathrm{ord}(g,x)})$, and whenever there is no exact leading-term cancellation one has matching Θ asymptotics. Condition (v) asks for some $c_{x,j} < c_x^{(0)}$ with $g = O(t^{c_{x,j}})$, which holds with $c_{x,j} = \mathrm{ord}(g, x)$ precisely when $\mathrm{ord}(g, x) < c_x^{(0)}$. □

C.3 Zero-penalty implies certificates

Proof of Proposition 3. By definition each $v_a(g) \geq 0$, and $V(g) = \sum_a v_a(g)$. Hence $V(g) = 0$ if and only if every score vanishes.

Implication. $v_{\text{mono}}(g) = 0$ iff $\overline{\partial F/\partial x} \leq 0$ for all $x \in \{N, D\}$, which is the monotonicity half of (7); likewise $v_{\text{conv}} = 0$ is equivalent to the convexity test and $v_{\text{irred}} = 0$ is equivalent to the floor test of Lemma 1 (and (iii) when $L_\infty > 0$). The remaining interval test $\overline{F} < \infty$ is not encoded as a soft score v_a: a DualInterval evaluation that completes with finite enclosures has in particular kept every division denominator and every pow_p argument away from 0 on the box, which is exactly the condition under which the operator set of Section 4.3 is C^∞ there, so (A5) / (vi) holds on $\mathcal{I}_x$ whenever the IA layer succeeds (Section 4.4(a)). $v_{\text{leaf}}(g) = 0$ iff both scale variables appear as leaves. $v_{\text{decay}}(g) = 0$ iff $\text{ord}(g, x) \leq c_x^{(0)} - \varepsilon_{\text{decay}} < c_x^{(0)}$, which is a strict strengthening of (v). Thus $V(g) = 0$ together with finite DualInterval enclosures implies certificates (a)–(b) (with the decay margin), and Theorem 1 applies stage by stage.

Measured gaps. Soundness of DualInterval enclosures yields $\partial F/\partial x \leq \overline{\partial F/\partial x}$ and $\partial^2 F/\partial x^2 \geq \underline{\partial^2 F/\partial x^2}$ on $\mathcal{I}_x$, so

$$\max(0, \partial F/\partial x) \;\leq\; \max(0, \overline{\partial F/\partial x})$$

pointwise on the box (and likewise for the convexity and floor gaps). Taking maxima over $x \in \{N, D\}$ shows that v_{mono}, v_{conv}, and v_{irred} upper-bound the corresponding true violations on the compact slice of $\widetilde{\mathbb{H}}$. The quantities $\text{ord}(g, x)$ and $\text{leaves}(g)$ are exact combinatorial reads of the tree, so v_{decay} and v_{leaf} are exact.

□

C.4 Boosting guarantee

Proof of Theorem 1. Induct on the stage index j. For $j = 0$, the claim is Proposition 1: $\hat{L}_0 \in \mathcal{S}$ with $L_\infty^{(0)} = E$ and the stated $c_x^{(0)}$.

Fix $j \geq 1$ and assume the claim holds through stage $j - 1$. Certificates (a)–(b) hold for $g = \hat{L}_j$, so by Lemma 1 conditions (i)–(iv) and the box half of (vi) hold on each certification box $\mathcal{I}_x$ (hence (A1)–(A5) on that compact slice of $\widetilde{\mathbb{H}}$), and by Lemma 2 condition (v) holds on the half-line. The box covers the observed grid; on $x > x_{\max}$, (v) together with $G = \hat{L}^{(j-1)} \in \mathcal{S}$ implies that the correction is negligible relative to the leading excess of G, so (A1)–(A2) of G persist for large x (the derivatives of an $O(t^c)$ term with $c < c_x^{(0)} < 0$ are o of those of the leading $t^{c_x^{(0)}}$ excess), and C^2 regularity (A5) extends by the operator set under the nonvanishing side condition of (vi) (Section 4.4(a)). Proposition 2 therefore yields $\hat{L}^{(j)} \in \mathcal{S}$ with the same joint floor $L_\infty^{(j)} = L_\infty^{(j-1)} = E$. The marginal-floor / rate calculation in the proof of Proposition 2 gives (10). □

C.5 Stage-0 Chinchilla admissibility

Proof of Proposition 1. Write the hyperparameter remainder

$$R(\mathbf{h}) \;\coloneqq\; \sum_{i=1}^{m} \frac{C_i}{\phi_i(h_i)^{\gamma_i}} + E,$$

so $\hat{L}_0 = AN^{-\alpha} + BD^{-\beta} + R(\mathbf{h})$. Every summand of R is strictly positive on $\widetilde{\mathbb{H}}$, hence $R(\mathbf{h}) > E > 0$. Shape axioms (A1), (A2), (A4), and (A5) are imposed only along $x \in \{N, D\}$ with the other coordinates held fixed; under that restriction R is constant in x.

(A1)–(A2). Fix $x = N$ and freeze $(D, h_1, \ldots, h_m)$. Then

$$\frac{\partial \hat{L}_0}{\partial N} \;=\; -\alpha A\, N^{-\alpha-1} \;<\; 0, \qquad \frac{\partial^2 \hat{L}_0}{\partial N^2} \;=\; \alpha(\alpha+1)\, A\, N^{-\alpha-2} \;>\; 0$$

for all $N \geq N_{\min} > 0$. The same identities with (A, α, N) replaced by (B, β, D) give (A1)–(A2) along D.

(A3). Set $L_\infty \coloneqq E > 0$. For every continuum configuration with $N, D \geq$ their minima (and hyperparameters in their H_h),

$$\hat{L}_0(\mathbf{h}) - E \;=\; \frac{A}{N^\alpha} + \frac{B}{D^\beta} + \sum_{i=1}^{m} \frac{C_i}{\phi_i(h_i)^{\gamma_i}} \;>\; 0,$$

so the joint-floor inequality of (A3) holds strictly on $\widetilde{\mathbb{H}}$. Along N with the other coordinates fixed,

$$L_N^\infty \;=\; \lim_{t\to\infty} \hat{L}_0\big|_{N=t} \;=\; \frac{B}{D^\beta} + R(\mathbf{h}) \;=\; \frac{B}{D^\beta} + \sum_{i=1}^{m} \frac{C_i}{\phi_i(h_i)^{\gamma_i}} + E,$$

which is finite and satisfies $L_N^\infty \geq E = L_\infty$ (and is strictly larger whenever $B > 0$ or $m \geq 1$). The marginal floor L_D^∞ is analogous.

(A4). With the same frozen coordinates, $\hat{L}_0\big|_{N=t} - L_N^\infty = A\, t^{-\alpha}$, hence

$$\lim_{t\to\infty} \frac{\log(\hat{L}_0\big|_{N=t} - L_N^\infty)}{\log t} \;=\; -\alpha \;=\; c_N^{(0)} \;<\; 0.$$

The D-axis calculation yields $c_D^{(0)} = -\beta < 0$.

(A5). On $[N_{\min}, \infty)$ the map $N \mapsto AN^{-\alpha}$ is C^∞ (hence C^2), the remaining terms are independent of N, and $\hat{L}_0(N_{\min}, \cdot) = AN_{\min}^{-\alpha} + BD^{-\beta} + R(\mathbf{h})$ is finite (and strictly larger than E by (A3)). The same holds with N replaced by D.

Thus $\hat{L}_0$ satisfies (A1)–(A5), i.e. $\hat{L}_0 \in \mathcal{S}$. □

Remark. If the hyperparameter block in (3) is replaced by nonnegative quadratic penalties in $\log \phi_{\mathrm{lr}}(\mathrm{lr})$ and $\log \phi_{\mathrm{wd}}(\mathrm{wd})$ (as in the 4D U-shaped baseline of Section 4.3), those terms remain constant along N and D. Whenever $A, B, E, \alpha, \beta > 0$, the same derivative and asymptotic calculations apply verbatim, so (A1)–(A5) continue to hold with the same $L_\infty = E$ and $c_x^{(0)}$.